

Complete comparison: solving the adjoint KKT linear system

We consider the linear system arising from the adjoint equations of a multi-stage optimal control problem:

$$\mathcal{L}_{\tau\tau} d\tau = -\frac{\partial L}{\partial \tau^*}, \quad d\tau \triangleq [\delta x, \delta \omega, \delta \lambda]$$

The matrix $\mathcal{L}_{\tau\tau}$ is block-tridiagonal in the natural stage ordering because stage k couples only to stages $k \pm 1$ through the dynamics.

Notation. Let $n = \dim(x)$, $m = \dim(\omega)$, $b = 2n + m$ the stage block size, N the horizon length, and $D = N \cdot b$ the total system dimension.

Tier 0 — Ignore all structure

Naive matrix inversion

Form $\mathcal{L}_{\tau\tau}^{-1}$ explicitly, then compute $d\tau = -\mathcal{L}_{\tau\tau}^{-1} \cdot \partial L / \partial \tau^*$.

Component	Cost
Assembly of $\mathcal{L}_{\tau\tau}$	$O(Nb^2)$
Explicit inversion	$O(D^3) = O(N^3b^3)$
Matrix-vector apply	$O(D^2) = O(N^2b^2)$
Memory (inverse is dense)	$O(D^2) = O(N^2b^2)$
Total	$O(N^3b^3)$

Worst on every axis. The inverse is dense even though $\mathcal{L}_{\tau\tau}$ is sparse, destroying the structure entirely. Also numerically worse — forming $\mathcal{L}_{\tau\tau}^{-1}$ squares the condition number relative to factor-and-solve. ****Never do this.****

Dense LDL^T on the full matrix

Factor $\mathcal{L}_{\tau\tau}$ as a generic symmetric indefinite matrix of size $D \times D$.

Component	Cost
Factorization	$O(D^3) = O(N^3b^3)$
Solve (per RHS)	$O(D^2) = O(N^2b^2)$
Memory	$O(D^2) = O(N^2b^2)$
Total	$O(N^3b^3)$

Better than explicit inversion (no inverse formed, better numerics), but still cubic in N . Any algorithm in Tier 0 is a non-starter for long horizons.

Tier 1 — Exploit the block-tridiagonal pattern (time coupling, matrix-level view)

These three algorithms all exploit the same fact: $\mathcal{L}_{\tau\tau}$ is block-tridiagonal in the natural stage ordering. All are backward-forward sweeps of length N . All achieve the same asymptotic complexity:

$$O(Nb^3) = O(N(n+m)^3)$$

They differ in constants, memory, and what byproducts they produce.

1a. Block-tridiagonal Gaussian elimination (block Thomas)

Block LU factorization, processing stage blocks sequentially. Forward elimination produces lower-block-bidiagonal L and upper-block-bidiagonal U ; back-substitution recovers the solution.

Component	Cost
Assembly	$O(Nb^2)$
Factorization	$O(Nb^3)$
Solve (per RHS)	$O(Nb^2)$
Memory	$O(Nb^2)$

Generic — doesn't require symmetry, doesn't require definiteness. Works but not the best fit for KKT systems because it doesn't exploit symmetry.

1b. Banded / block-tridiagonal LDL^T factorization

Symmetric indefinite factorization exploiting the block-tridiagonal pattern. Produces $\mathcal{L}_{\tau\tau} = LDL^T$ with L block lower-bidiagonal and D block-diagonal with stage-sized indefinite blocks.

Component	Cost
Assembly	$O(Nb^2)$
Factorization	$\frac{1}{2}$ the flops of block Thomas, still $O(Nb^3)$
Solve (per RHS)	$O(Nb^2)$
Memory	$O(Nb^2)$

Exploits symmetry $\rightarrow$ roughly half the arithmetic of 1a, and the natural choice when you hand the problem to a general-purpose sparse linear-algebra library. But still operates on the assembled matrix.

1c. Riccati recursion (KKT-structure-aware)

Operates directly on the stage data $(F_k, G_k, L_k^{xx}, L_k^{x\omega}, L_k^{\omega\omega}, \partial L / \partial x_k, \dots)$ without assembling $\mathcal{L}_{\tau\tau}$. Backward sweep propagates (P_k, p_k) ; within each stage, $\delta\omega_k$ and $\delta\lambda_k$ are eliminated analytically before advancing. Forward sweep recovers $(\delta x_k, \delta\omega_k, \delta\lambda_k)$.

Component	Cost
Assembly	none
Factorization + solve (combined into the sweep)	$O(Nb^3)$ with the smallest constants
Memory	$O(N(n^2 + nm))$ — carries only $P_k \in \mathbb{R}^{n \times n}$ and gain $K_k \in \mathbb{R}^{m \times n}$ per stage
Byproducts	cost-to-go Hessians P_k , feedback gains K_k , cost-to-go gradients p_k

Same asymptotic order as 1a and 1b, but **lower constants** (2–5× faster in practice), **lower memory** (no full stage blocks stored), **no matrix assembly**, and **useful byproducts** that matter for the outer optimization loop (warm-starting, feedback policy, sensitivity).

Equivalence and practical differences among Tier-1 methods

The three Tier-1 methods are **mathematically equivalent** on this KKT system: they compute the same unique solution $d\tau$ via the same backward-forward sweep along the time axis, performing arithmetically the same Schur-complement updates (up to relabeling) in the same order. The Riccati quantities (P_k, K_k, p_k) are exactly the block pivots, off-diagonal factors, and reduced right-hand sides that appear inside the block Thomas / banded LDL^T factorization — just given a control-theoretic interpretation (cost-to-go Hessian, feedback gain, cost-to-go gradient). If you run all three with the same numerical precision, they produce the same $d\tau$ up to roundoff.

They all share the same asymptotic complexity $O(Nb^3)$, and the same memory order $O(Nb^2)$ at worst. The differences show up only in the *constants* and the *total wall-clock cost* once you account for every stage of the pipeline, not just the factorization flops:

Cost component	Block Thomas (1a)	Banded LDL^T (1b)	Riccati (1c)
Matrix assembly	required, $O(Nb^2)$	required, $O(Nb^2)$	none
Exploits symmetry	no	yes ($\sim \frac{1}{2}$ flops of 1a)	yes
Eliminates $\delta\omega_k, \delta\lambda_k$ within stage	no — carries full $b \times b$ blocks	no — carries full $b \times b$ blocks	yes — carries only $n \times n$ and $m \times n$ blocks
Active working set per stage	$O(b^2) = O((n + m)^2)$	$O(b^2)$	$O(n^2 + nm)$
Typical wall-time ratio vs 1c	3-5× slower	2-3× slower	baseline
Produces feedback gains K_k	no (would need extraction)	no (would need extraction)	yes, for free
Fits into SQP / iLQR / DDP outer loop	awkwardly (re-extract)	awkwardly (re-extract)	natively

Takeaway on equivalence vs. cost. The three methods are the same algorithm dressed in different vocabularies and exploiting successively more of the problem's structure — block Thomas sees only the block-tridiagonal sparsity; banded LDL^T additionally sees the symmetry; Riccati additionally sees the internal KKT structure of each stage. Moving down the list doesn't change the asymptotic complexity or the answer, but it strictly reduces the work done — no redundant arithmetic from ignoring symmetry, no matrix to assemble, no within-stage variables carried further than needed. The total computational cost therefore follows the strict ordering

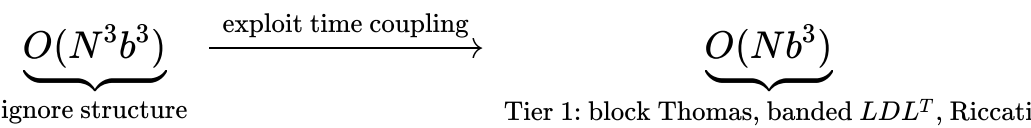
$$\text{cost}(\text{block Thomas}) > \text{cost}(\text{banded } LDL^T) > \text{cost}(\text{Riccati})$$

even though all three sit at $O(Nb^3)$ and all three return the identical $d\tau$.

Summary table

Method	Structure used	Factorization	Solve	Memory	Notes
Explicit inversion	none	$O(N^3b^3)$	$O(N^2b^2)$	$O(N^2b^2)$	Worst on every axis
Dense LDL^T	none	$O(N^3b^3)$	$O(N^2b^2)$	$O(N^2b^2)$	No sparsity awareness
Block Thomas (Gaussian)	block-tridiagonal	$O(Nb^3)$	$O(Nb^2)$	$O(Nb^2)$	Generic, ignores symmetry
Banded LDL^T	block-tridiagonal + symmetry	$O(Nb^3)$	$O(Nb^2)$	$O(Nb^2)$	$\sim\frac{1}{2}$ constants of block Thomas
Riccati recursion	block-tridiagonal + symmetry + stage KKT structure	$O(Nb^3)$	$O(Nb^2)$	$O(N(n^2 + nm))$	Lowest constants; no assembly; feedback gains free

The hierarchy in one picture



Moving from Tier 0 to Tier 1 — from cubic in N to linear in N — is the **only** order-of-magnitude improvement available, and it comes entirely from recognizing the temporal structure. Within Tier 1, the three methods are algorithmically equivalent and return the identical solution; they differ only in how much of the structure beyond block-tridiagonality they additionally exploit, which translates into constant-factor differences in total cost.

Practical verdict

- **Never** use Tier 0 unless N is trivially small.

- **Block Thomas** and **banded LDL^T** are appropriate when you're using a general-purpose sparse linear algebra stack and just want the KKT system solved as a black box.
- **Riccati** is the implementation of choice in any serious optimal-control code (MPC, iLQR, DDP, trajectory optimization): same $O(Nb^3)$ order as the other Tier-1 methods, but with lower constants, no assembly, less memory, and the cost-to-go / feedback structure handed to you as a byproduct.

The structural insight is the big one — going from $O(N^3)$ to $O(N)$ in the horizon is the whole ball game, and it comes from the time coupling of the KKT conditions being block-tridiagonal. Riccati just happens to be the most economical way to execute on that insight.

王秉亨

18-Apr-2026